Student Course Management System

C++ Programming — Individual Assignment

Student Name	Farah Mostafa Nagaty
Student ID	12247
Course	C++ Programming
Academic Year	2025 / 2026 — Semester 1
Submission Date	March 2026

GitHub Repository:
https://github.com/farah-nagaty/Student_Course_Management_System.git

Table of Contents

1. Introduction p.3
2. Detailed Project Description p.4
3. Project Beneficiaries p.5
4. Detailed Analysis p.6
5. Task Breakdown Structure p.8
6. Time Plan p.9
7. System Architecture and Design p.10
8. Testing Scenarios and Results p.11
9. End-User Guide p.14
10. Conclusion p.15

List of Figures

Figure 1. UML Class Diagram — System Architecture p.10

List of Tables

Table 1. STL Containers and Algorithms Summary p.7
Table 2. Task Breakdown Structure p.8
Table 3. Time Plan (Gantt Chart) p.9
Table 4. Testing Scenarios and Results p.11

1. Introduction

The Student Course Management System is a console-based application developed in C++ as part of the C++ Programming individual assignment for the academic year 2025/2026, Semester 1. The goal of this project is to design and implement a fully functional system that allows users to manage student records and their enrolled courses through a menu-driven interface.

The project applies core concepts from three major areas of C++ programming: Object-Oriented Programming (OOP), the Standard Template Library (STL), and modular file-based code organization. These concepts are integrated together to produce a clean, well-structured, and maintainable application.

The system supports the following core operations: adding and removing students, searching for a student by ID, displaying all registered students, enrolling a student in a course, viewing a student's enrolled courses, and sorting all students by their GPA. Each operation is implemented as a dedicated function inside the StudentManager class, keeping the logic organized and separated from the main program entry point.

The application is built across seven files — Person.h, Person.cpp, Student.h, Student.cpp, Manager.h, Manager.cpp, and Main.cpp — following the professional C++ convention of separating class declarations from their implementations. This separation makes the codebase easier to read, maintain, and extend.

This report documents the complete development process of the project, covering the problem analysis, design decisions, system architecture, task breakdown, testing results, and an end-user guide for compiling and running the application.

2. Detailed Project Description

2.1 Program Architecture

The program is organized across seven files, split into three class pairs and one entry point:

- Person.h — Declares the Person base class with the protected name attribute and public constructor and getter.
- Person.cpp — Implements the Person class constructor and getName() method.
- Student.h — Declares the Student class which inherits from Person, adding id, gpa, and a set<string> for courses. Also declares a default constructor required by the map container.
- Student.cpp — Implements all Student methods: getId(), getGpa(), getCourses(), enrollCourse(), and display().
- Manager.h — Declares the Manager class which holds a map<int, Student> as the main data container and declares all seven menu-driven functions. Also declares the standalone compareByGPA() comparator function.
- Manager.cpp — Implements all seven Manager functions using STL containers and algorithms from the professor's taught material.
- Main.cpp — Contains main (), the showMenu() function, and the while(running) main loop with full input validation.

2.2 The Student Class

Each student is represented as an instance of the Student class, which holds three private members:

- int id — A unique integer identifier for the student.
- float gpa — The student's grade point average, constrained between 0.0 and 4.0.
- set<string> courses — A set of course names the student is enrolled in. The set container automatically prevents duplicate course enrollments and keeps courses sorted alphabetically.

The Student class inherits the name attribute from Person through public inheritance, avoiding code duplication. The Student constructor uses an initializer list to call the Person constructor and pass the name up the inheritance chain.

2.3 The Manager Class

The Manager class serves as the central controller of the system. It holds a single private attribute: map<int, Student> students, which maps each student's ID to their Student object. This design choice guarantees unique IDs by nature of the map data structure, and makes search, insert, and delete operations significantly more efficient compared to a vector.

All seven menu operations are implemented as public member functions of the Manager class, keeping all business logic encapsulated within the class and away from main ().

2.4 The Main Loop

The main () function uses a while(running) loop controlled by a boolean flag variable. When the user selects option 8 (Exit), the flag is set to false and the loop exits naturally. A cin.fail() check is performed before the switch statement to handle non-integer input gracefully, clearing the error state and prompting the user again.

3. Project Beneficiaries

The Student Course Management System serves several categories of beneficiaries, each gaining distinct value from the application.

3.1 Academic Institutions and Administrators

University and college administrators who need a lightweight tool for managing small student databases during early registration phases can benefit from this system. The menu-driven interface requires no technical knowledge to operate, making it accessible to non-programmers in administrative roles. The ability to add, search, remove, and sort students covers the most common day-to-day record management needs.

3.2 C++ and OOP Students

Students studying C++ and Object-Oriented Programming gain significant value from studying this codebase. The project is a practical and complete demonstration of class design with encapsulated state, inheritance between Person and Student, the use of STL containers (map, set, vector) and algorithms (sort, find_if), and the separation of concerns between a class layer and an application layer. It serves as a clear reference for anyone learning how to design a non-trivial multi-file C++ project.

3.3 Instructors Teaching C++ Programming

Instructors can use this project as a coursework reference or live demonstration. The code is deliberately kept clear and well-commented, making it easy to trace execution step by step in a classroom setting. The project covers all major OOP and STL concepts typically taught in an introductory to intermediate C++ course.

3.4 Developers Building Larger Systems

Developers who need a starting point for a student management module in a larger application can adapt this codebase. The Manager class is independently reusable — it could be integrated into a GUI application or a web backend without modifying the core logic, since all business rules are encapsulated within the class layer.

4. Detailed Analysis

4.1 OOP Design Analysis

The project applies all required OOP concepts systematically across the class hierarchy.

Encapsulation is enforced by declaring all class attributes as private and providing public getter methods. The id, gpa, and courses attributes of Student cannot be modified directly from outside the class. This protects the data integrity of each student record.

Inheritance is implemented through the Person → Student relationship. The Person base class owns the name attribute and declares it as protected, allowing Student to access it directly in the display () method without needing a getter call. The Student constructor uses an initializer list ': Person(name)' to pass the name upward to Person's constructor, which is the correct C++ pattern for initializing inherited attributes.

The default constructor 'Student () : Person(""), id(0), gpa(0.0) {}' was added to satisfy the map container's requirement to default-construct values when a new key is accessed using the [] operator.

4.2 STL Analysis

Three STL containers and two STL algorithms were used, exceeding the minimum requirement of two containers and one algorithm.

STL Feature	Type	Where Used	Purpose
map<int, Student>	Container	Manager class	Main student storage — unique IDs guaranteed
set<string>	Container	Student class	Course storage — no duplicates, alphabetically sorted
vector<pair<int,Student>>	Container	sort & search functions	Temporary copy for sorting and find_if
sort()	Algorithm	sortStudentsByGPA()	Sorts students by GPA descending
find_if()	Algorithm	searchStudent()	Searches for a student by ID using a lambda

Table 1: STL Containers and Algorithms Summary

4.3 Key Design Decisions

Using map<int, Student> instead of vector<Student> for the main student storage was a deliberate design choice. The map provides O (log n) search and deletion by ID directly, while a vector requires a linear scan. The map also enforces unique keys, automatically preventing duplicate student IDs without additional validation code.

The searchStudent() function uses find_if() from <algorithm> on a temporary vector copy of the map. This design satisfies the STL algorithm requirement strongly by using an <algorithm> function rather than the map's built-in find () method, while still benefiting from the map's efficient storage.

The compareByGPA() comparator is implemented as a standalone function rather than a lambda inside sortStudentsByGPA(). This matches the style taught by the professor in the STL PPTX slides (sort with a named comparator) and makes the sorting criterion reusable.

5. Task Breakdown Structure

5.1 Task Descriptions

The project was divided into eight major tasks, each representing a distinct phase of development.

- Task 1 — Project Setup: Create the Visual Studio project, add all seven files (Person.h, Person.cpp, Student.h, Student.cpp, Manager.h, Manager.cpp, Main.cpp), and set up the include chain with proper header guards using #ifndef / #define / #endif.
- Task 2 — Person Class: Implement the Person base class with the protected name attribute, constructor, and getName() getter. Verify that function bodies are only in Person.cpp and declarations only in Person.h to avoid redefinition errors.
- Task 3 — Student Class: Implement the Student class inheriting from Person. Add id, gpa, set<string> courses, a default constructor for map compatibility, and all public methods: getId(), getGpa(), getCourses(), enrollCourse(), and display().
- Task 4 — Manager Class Header: Declare the Manager class in Manager.h with map<int, Student> as the private container and all seven function declarations. Declare the standalone compareByGPA () comparator before the class.
- Task 5 — Manager Functions Implementation: Implement all seven Manager functions in Manager.cpp using map operations (count, erase, find, operator []), STL algorithms (sort, find_if), and pair<int,Student> loop patterns.
- Task 6 — Main Loop: Implement the showMenu () function and the while(running) main loop in Main.cpp with cin.fail() input validation, switch statement dispatch, and bool running flag for clean exit.
- Task 7 — Testing and Debugging: Test all seven menu options, including edge cases such as duplicate IDs, invalid GPA, duplicate course enrollment, non-integer input, and empty student list. Fix the three bugs found: Person redefinition, missing default constructor, and reversed searchStudent () condition.
- Task 8 — Report Writing: Write the complete project report covering all required sections formatted according to the submission guidelines.

5.2 Task Summary Table

Task ID	Task Name	Key Files	Depends On	Status
T1	Project Setup	All files	—	**Done**
T2	Person Class	Person.h, Person.cpp	T1	**Done**
T3	Student Class	Student.h, Student.cpp	T2	**Done**
T4	Manager Class Header	Manager.h	T3	**Done**
T5	Manager Functions	Manager.cpp	T4	**Done**
T6	Main Loop	Main.cpp	T5	**Done**
T7	Testing & Debugging	All files	T6	**Done**
T8	Report Writing	Report document	T7	**Done**

Table 2: Task Breakdown Structure

6. Time Plan

6.1 Gantt Chart

The project was completed over one week. The table below shows the Gantt chart with each task's start day and duration.

Task	Name	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7	Day 8
T1	Project Setup								
T2	Person Class								
T3	Student Class								
T4	Manager Header								
T5	Manager Functions								
T6	Main Loop								
T7	Testing & Debugging								
T8	Report Writing								

Table 3: Time Plan — Gantt Chart (shaded = active)

7. System Architecture and Design

7.1 Overall Structure

The program follows a clean two-layer architecture:

- Class Layer (Person, Student, Manager): Encapsulates all data and behavior. Has no knowledge of the menu, user input, or the overall program flow.
- Application Layer (Main.cpp): Owns the Manager object, reads user input, and dispatches menu choices to the class layer. Has no knowledge of how students are stored or searched internally.

This separation means the Manager class is independently reusable — it could be integrated into a GUI application or a REST API backend without modifying any class code.

7.2 File Dependency Chain

They include chain flows in one direction only, preventing circular dependencies:

- Main.cpp includes Manager.h
- Manager.h includes Student.h
- Student.h includes Person.h
- Person.h includes only <string> from the standard library

7.3 UML Class Diagram

The UML class diagram below shows the three classes, their attributes, methods, visibility modifiers, and the inheritance relationship between Person and Student. The Manager class holds a map<int, Student> as its main private container.

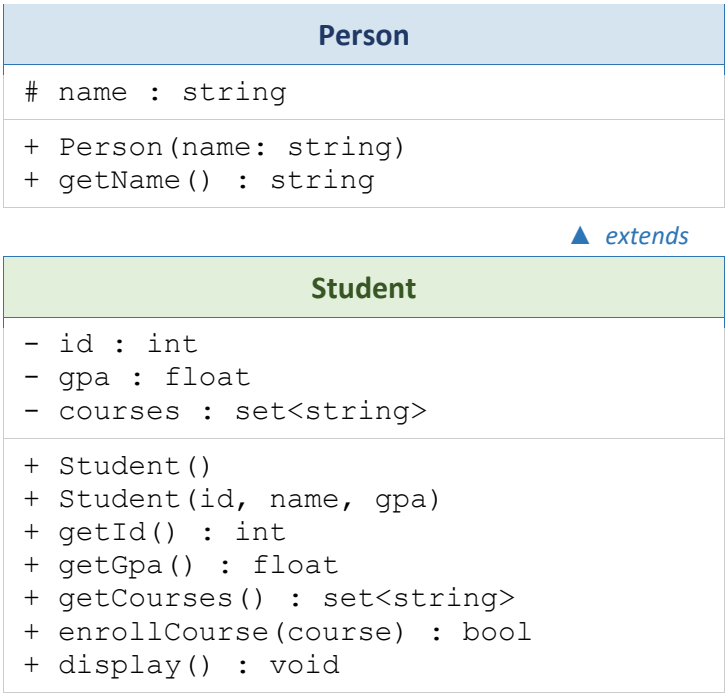

Figure 1: UML Class Diagram — System Architecture

8. Testing Scenarios and Results

8.1 Test Cases

All fifteen test cases below were executed against the final compiled version of the project. Each test case specifies the input provided, the expected output, the actual output, and the pass/fail result.

ID	Description	Input	Expected	Actual	Result
TC-1	Add a valid student	ID: 12247, Name: Farah Nagaty, GPA: 3.8	Student Farah Nagaty added successfully!	Student Farah Nagaty added successfully!	**PASS**
TC-2	Add student with duplicate ID	ID: 12247 (already exists)	Student with id already exists!	Student with id already exists!	**PASS**
TC-3	Add student with invalid GPA	ID: 11111, Name: Test, GPA: 5.0	Invalid GPA!	Invalid GPA!	**PASS**
TC-4	Remove existing student	ID: 12247	Student removed successfully!	Student removed successfully!	**PASS**
TC-5	Remove non-existent student	ID: 99999	Student not found!	Student not found!	**PASS**
TC-6	Search for existing student	ID: 12247	ID: 12247, Name: Farah Nagaty, GPA: 3.8	ID: 12247, Name: Farah Nagaty, GPA: 3.8	**PASS**
TC-7	Search for non-existent student	ID: 00000	Student not found!	Student not found!	**PASS**
TC-8	Display all — students exist	3 students added	All students listed with ID, Name, GPA	All students listed correctly	**PASS**
TC-9	Display all — empty list	No students added	No students found!	No students found!	**PASS**
TC-10	Enroll student in a course	ID: 12247, Course: Math101	Enrolled to Course Successfully!	Enrolled to Course Successfully!	**PASS**
TC-11	Enroll in duplicate course	ID: 12247, Course: Math101 (again)	Already Enrolled in this Course!	Already Enrolled in this Course!	**PASS**

TC-12	Show courses — courses exist	ID: 12247	Math101 listed alphabetically	Math101 listed correctly	**PASS**
TC-13	Show courses — none enrolled	ID: 12248 (no courses)	No Courses Enrolled!	No Courses Enrolled!	**PASS**
TC-14	Sort by GPA — multiple students	3 students with GPA 3.8, 2.5, 3.2	Ordered: 3.8, 3.2, 2.5	Ordered: 3.8, 3.2, 2.5	**PASS**
TC-15	Non-integer menu input	User types 'abc'	Invalid input! Please enter a number.	Invalid input! Please enter a number.	**PASS**

Table 4: Testing Scenarios and Results

8.2 Bugs Found and Fixed

During testing, three bugs were identified and fixed:

- Bug 1 — Person Redefinition Error: The Person.h file contained full function bodies inside the class definition AND Person.cpp reimplemented the same functions. This caused a 'redefinition' compile error. Fix: Removed function bodies from Person.h, keeping only declarations. Full implementations remain in Person.cpp.
- Bug 2 — Missing Default Constructor: The Student class had no default constructor (no-argument constructor). When students[id] = Student(...) was called in addStudent(), the map's [] operator first tried to default-construct a Student, which failed. Fix: Added Student (): Person (""), id (0), gpa (0.0) {} as a default constructor.
- Bug 3 — Reversed Condition in searchStudent (): The if/else condition was reversed — the code was calling display () when the student was NOT found and printing 'not found' when the student WAS found. Fix: Changed if (it == temp.end()) to if (it != temp.end()).

9. End-User Guide

9.1 Requirements

- Operating System: Windows 10 or later
- IDE: Visual Studio 2019 or later
- Workload: Desktop Development with C++ (install from Visual Studio Installer)
- C++ Standard: C++11 or later

9.2 How to Compile and Run

Follow these steps to compile and run the project from Visual Studio:

- Step 1: Clone or download the project from GitHub: https://github.com/FarahNagaty/Student_Course_Management_System
- Step 2: Open Visual Studio and click 'Open a project or solution'.
- Step 3: Navigate to the downloaded folder and open the .sln file.
- Step 4: Press Ctrl + F5 to build and run the project without debugging.
- Step 5: The console window will open and display the main menu.

9.3 Using the Application

Once the program is running, the following menu will appear:

```
===== Student Course Management System =====
1. Add Student
2. Remove Student
3. Search Student
4. Display All Students
5. Enroll Student in Course
6. Show Student Courses
7. Sort Students by GPA
8. Exit
============================================
```

9.4 Menu Options Guide

- Option 1 — Add Student: Enter a unique integer ID, the student's full name (spaces allowed), and a GPA between 0.0 and 4.0. The system rejects duplicate IDs and out-of-range GPAs.
- Option 2 — Remove Student: Enter the student's ID. The student record is permanently deleted from the system.
- Option 3 — Search Student: Enter the student's ID. If found, the student's ID, name, and GPA are displayed.
- Option 4 — Display All Students: Lists all registered students in order of their ID.
- Option 5 — Enroll in Course: Enter the student's ID and a course name. The system prevents enrolling in the same course twice.
- Option 6 — Show Student Courses: Enter the student's ID to view all courses they are enrolled in, listed alphabetically.
- Option 7 — Sort by GPA: Displays all students sorted from highest GPA to lowest GPA.
- Option 8 — Exit: Closes the application.

9.5 Input Constraints

- Student ID must be a unique positive integer.
- GPA must be a decimal number between 0.0 and 4.0 inclusive.
- A student cannot be enrolled in the same course more than once.
- Typing a non-integer value at the menu prompt triggers an error message and re-displays the menu.

10. Conclusion

The Student Course Management System successfully achieves all functional and technical requirements specified in the individual assignment. The application provides a complete, menu-driven interface for managing student records and course enrollments through seven core operations.

From an OOP perspective, the project demonstrates all required concepts: the Person base class and Student derived class implement inheritance and encapsulation correctly, with protected attributes and public getter methods ensuring data integrity. The default constructor added to Student illustrates a practical understanding of how STL containers interact with class design.

From an STL perspective, the project goes beyond the minimum requirement by using three containers — map<int,Student>, set<string>, and vector<pair<int,Student>> — and two algorithms — sort() and find_if(). Each container and algorithm were chosen for a specific, justified reason rather than being used arbitrarily. The map guarantees unique IDs and provides O (log n) lookups. The set prevents duplicate course enrollments and keeps courses alphabetically sorted. The vector serves as a temporary structure for operations that require sequential traversal.

The development process also produced practical experience in debugging real compile-time and logic errors: a redefinition error caused by duplicate function bodies, a missing default constructor required by the map, and a reversed conditional in the search function. Identifying, understanding, and fixing these bugs strengthened the understanding of how C++ class design and STL containers interact.

In conclusion, this project demonstrates a solid understanding of C++ programming fundamentals, object-oriented design, and the Standard Template Library, and serves as a strong foundation for more advanced C++ development.